

Shell Scripting-3 (cntd..)

Dr. Soma Ghosh
gsn.comp@coeptech.ac.in

File Test Operators

- `-b file` • `[ -b $file ]` <Checks if file is block special
>
- `-c file` • `[ -c $file ]` <character special file>
- `-d file` • `[ -d $file ]` < checks if directory>
- `-f file` • `[ -f $file ]` < checks ordinary file>
- `-r file` • `[ -r $file ]` <checks if file is readable>
- `-w file` • `[ -w $file ]` <checks if file is writable>
- `-x file` • `[ -x $file ]` <checks if file is executable>
- `-s file` • `[ -s $file ]` <checks if file size gt 0>
- `-e file` • `[ -e $file ]` <checks if file exists>

Loops

- While loop
- For loop
- Until loop

While loop example

```
#!/bin/sh
a=0
while [ "$a" -lt 10 ]
do
    b="$a"
    while [ "$b" -ge 0 ]
    do
        printf "%s" "$b"
        b=$(expr "$b" - 1)
    done
    printf "\n"
    a=$(expr "$a" + 1)
done
```

until loop example

```
#!/bin/sh
```

```
a=20
```

```
until [ $a -lt 10 ]
```

```
do
```

```
    echo $a
```

```
    a=$((a - 1))
```

```
done
```

break stmt example

```
#!/bin/sh
a=0
while [ "$a" -lt 10 ]
do
    echo "$a"
    if [ "$a" -eq 5 ]
    then
        break
    fi
    a=$(expr "$a" + 1)
done
```

for and break n example

```
#!/bin/bash
for x in 1 2 3
do
    for y in 0 5
    do
        if [ "$x" -eq 2 -a "$y" -eq 0 ]
        then
            break 2
        else
            echo "$x $y"
        fi
    done
done
```

continue

- continue
- continue n

caseesac example

```
#!/bin/sh
echo "enter a character"
read char
case $char in
    [a-z]) echo " you entered a small case letter"
    [A-Z]) echo "you entered a capital letter"
    [0-9]) echo " you entered a numeric"
    ?) echo " you entered a special character"
    *0) echo " you entered more than one character"
esac
```

escape sequences used with echo

- `\\`
- `\a`
- `\b`
- `\c`
- `\n`
- `\r`
- `\t`
- `\v`
- backslash
- alert (bell)
- backspace
- suppress trailing newline
- new line
- carriage return
- horizontal tab
- vertical tab

example escape sequences

```
#!/bin/sh
```

```
a=10
```

```
echo -e "value of a is $a \n"
```

Note:-

1. try executing without -e
2. try executing other escape sequences given in previous slide

I/O redirection and piping

- Redirection facility lets us connect Unix commands to files.
- Piping facility lets us connect commands to commands

example of redirection operator

```
$ touch text.txt
```

```
$ echo line1 > text.txt      (output  
redirection)
```

```
$ cat text.txt
```

```
$ echo line2 >> text.txt    (append)
```

```
$ cat text.txt
```

```
$ wc -l < text.txt          (input  
redirection)
```

example of piping operator

```
$ ls | wc -l
```

o/p of ls becomes i/p to wc which counts number of lines it receives as i/p

```
$ ls | wc -l > countfile
```

```
$ who | sort
```